Contribution Title

Lucas Lacampagne

CentraleSupélec

GitHub link : https://github.com/lucas-lacampagne/assignment_RL

1 Question 1

1.1 Experimental setup

We use the environment initiate environment *TextFlappyBird-v0* with the following parameters : $height = 15, width = 20, pipe_gap = 4$.
I implemented a function to visualize q-tables and two methods to evaluate and visualize how the agent performs after training, both using greedy action selection by default:

- evaluate_agent plays num_tries games of max_steps number of steps maximum and keep track of the number of steps and the scores obtained. Then, I get stats over this data using Pandas.
- visualize_agent allows to render games, even without using the *screen* version of the environment

1.2 Agents

I implemented two agents : a MonteCarlo and a Sarsa one. They both use epsilon-greedy action selection during training. In both cases, I performed grid search to tune their hyperparameters. I evaluate their performance based on their average time alive and their average score over $num_tries = 100$ games of $max_steps = 10000$ steps maximum. We display only the best results but the testing has been made over :

- Sarsa Agent : $\alpha \in \{0.1, 0.36, 0.63, 0.9\}, \gamma \in \{0.1, 0.4, 0.7, 1\}, \lambda \in \{0.1, 0.4, 0.7, 1\}$
- MonteCarlo Agent : $\gamma \in \{0.1, 0.4, 0.7, 1\}$

We can see that Sarsa is pretty bad, even with some tuned parameters. Here's a histogram that shows its stats :

MonteCarlo agent obtains a perfect performance, scoring 999 each attempt.

2 Question 2

TextFlappyBird-v0's state are the coordinates of the bird while *TextFlappyBird-screen-v0*'s one is the entire screen. Also, the state space of the second is far greater than the first.

α	γ	λ	Average time alive	Average score
0.36	0.70	0.70	739.54	73.00
0.36	0.70	0.40	742.73	73.33
0.36	0.10	1.00	766.22	75.70
0.36	0.10	0.40	773.71	76.48
0.63	0.70	1.00	881.81	87.22
0.36	0.40	0.70	970.04	95.99

Table 1. Grid search over α, γ, λ for Sarsa Agent

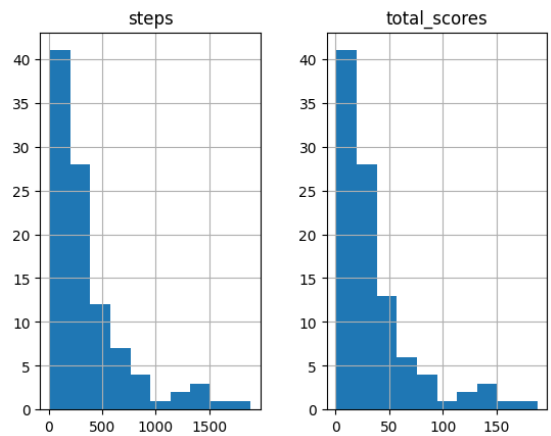

Fig. 1. Histogram showing Sarsa's results

γ	Average time alive	Average score
0.1	107.54	9.46
0.5	854.00	84.19
0.9	10000.00	999.00

Table 2. Grid search over γ for MonteCarlo Agent

3 Question 3

With an implementation of the original flappy bird game environment3 available, can the same agents be used? If I want my agents, who were trained on the first one, to be able to run over the second, I would need a feature extractor.

4 Question 4

Given a trained agent on a specific configuration of the TFB environment (e.g., height=15, width=20, pipe gap=4), how well does the trained agent perform on a different level configuration? We still consider the same MonteCarlo agent as before, trained in the same conditions. Here are the results for

Table 3. Sensitivity Analysis: Performance impact of Environment Configuration Changes. Baseline results are highlighted in the center row.

Parameter	Value	Mean Steps	Max Steps	Mean Score	Max Score
Baseline	*Default*	*10,000.00*	*10,000*	*999.00*	*999*
Pipe Gap	2	28.18	93	1.62	8
	6	10,000.00	10,000	999.00	999
Height	8	10,000.00	10,000	999.00	999
	20	157.25	713	14.82	70
Width	10	102.69	466	19.35	92
	30	4.00	4	0.00	0

5 Question 5

A comprehensive presentation of the results and the performance of the different agents. You already got the results in question 1, during the tuning of the agents. We kept the parameters that gave the best performance. Here we add plots for the actions-value functions.

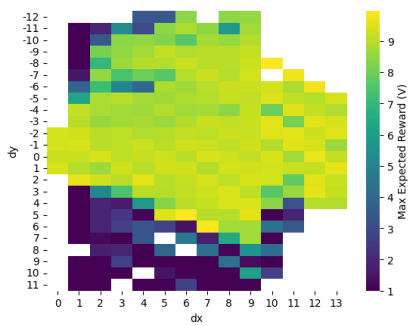

Fig. 2. Q-Table : opinion (y) vs action (x)

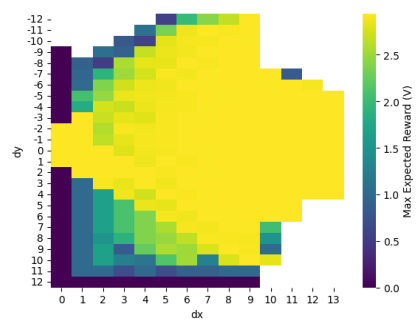

Fig. 3. Q-Table : opinion (y) vs action (x)